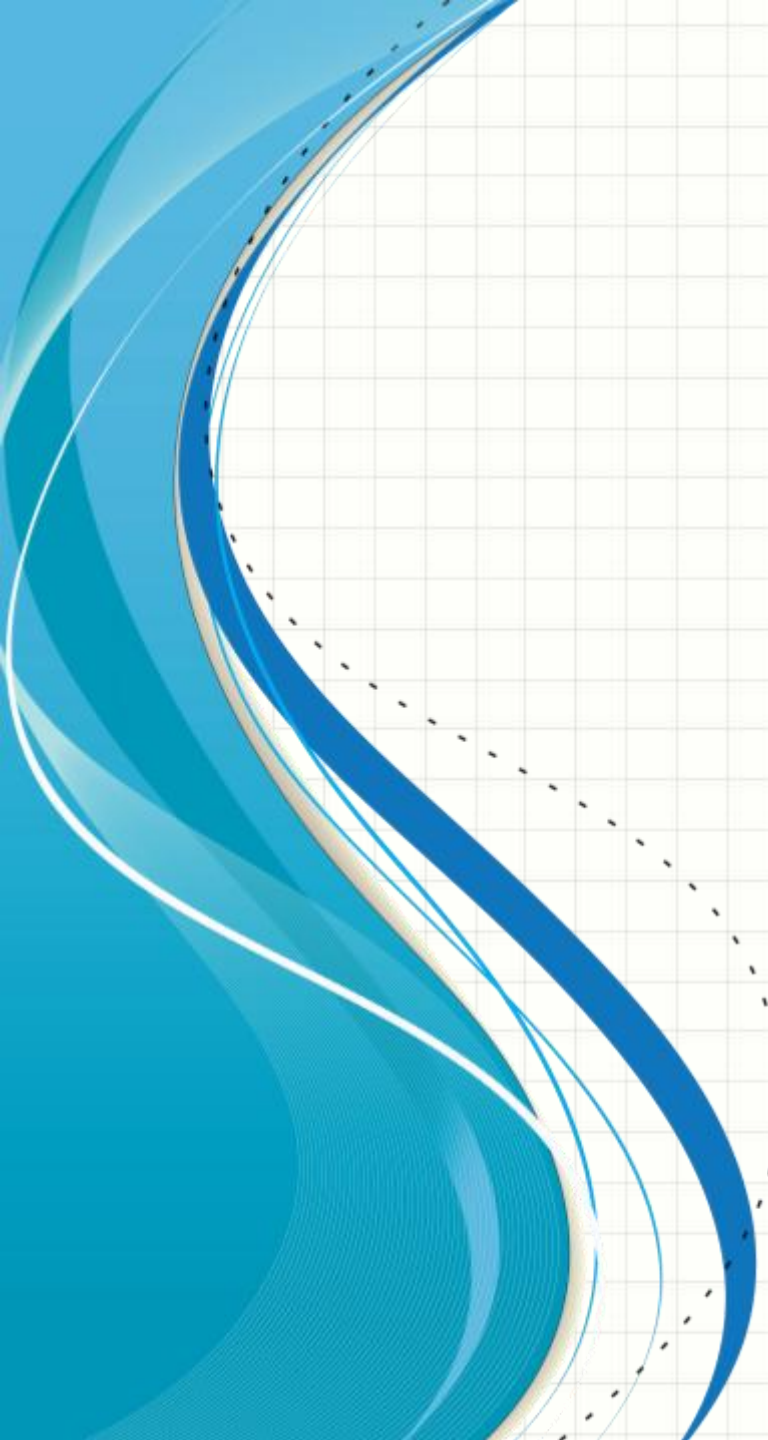




CHAPTER # 2

BOOLEAN ALGEBRA

AND

LOGIC GATES

Lecturer: Noman Al Hassan

Email: noman44m@hotmail.com

Objective of Lecture

After studying this chapter, you should be able to know

- Logic Gates.
 - Types of Logic Gates.
 - I/O Levels.
 - Truth Tables.
 - Negative positive logic.
 - Binary logic.
- Boolean Algebra.
 - Implementation of Boolean Function into Logic Circuit.
 - Manipulate Boolean Expressions and Simplify.

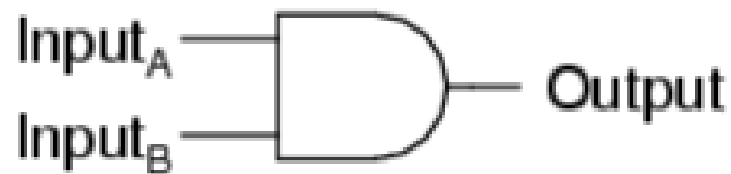
Logic Gate

- Boolean algebra is an algebra that deals with binary variables and logic operations. Boolean functions may be practically implemented by using electronic gates.
- A **logic gate** is an elementary building block of a digital circuit.
- Most **logic gates** have two inputs and one output. The two values of binary variables may be equivalently referred to as 0 and 1 or False (0) and True (1) or as Low (0) and High(1).
- Digital systems are said to be constructed by using logic gates. These gates are the AND, OR, NOT, NAND, NOR, EXOR and EXNOR gates.

Types of Logic Gates

- The AND gate is an electronic circuit that gives a **high output (1)** **only if all its inputs are high**.
- A dot (.) is used to show the AND operation.

AND gate



A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

Types of Logic Gates

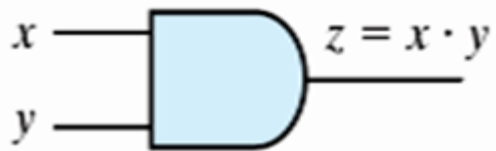
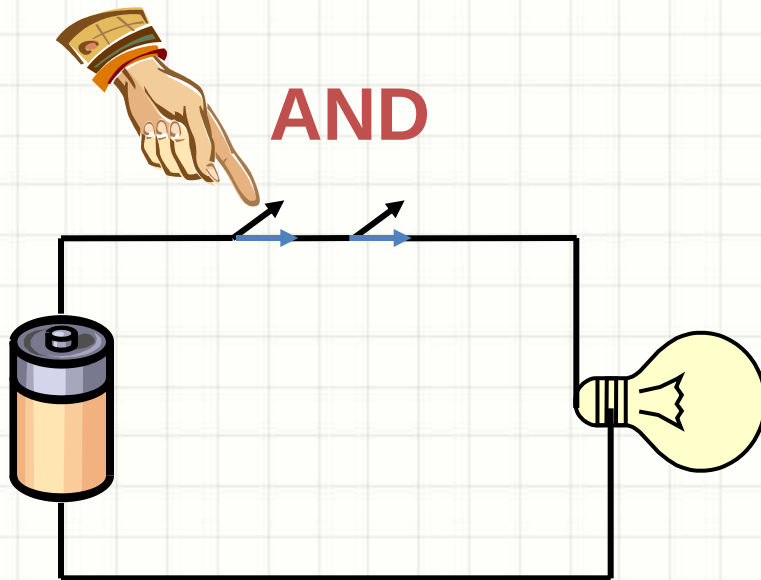
- The OR gate is an electronic circuit that gives a **high output (1)** if **one or more of its inputs are high**.
- A plus (+) is used to show the OR operation.

OR gate

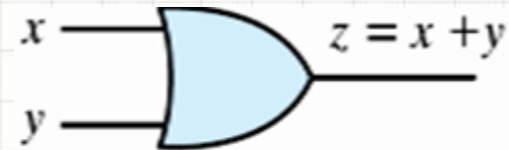
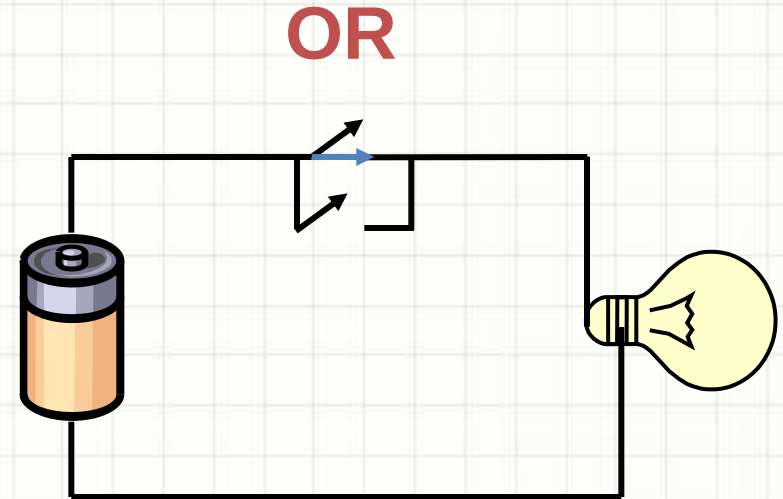


A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1

Example: Switching Circuits



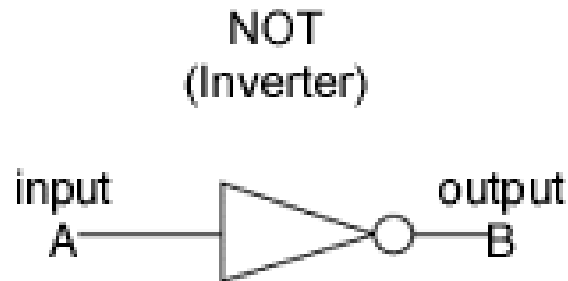
(a) Two-input AND gate



(b) Two-input OR gate

Types of Logic Gates

- The NOT gate is an electronic circuit that produces **an inverted version of the input**.
- It is also known as an *inverter*.

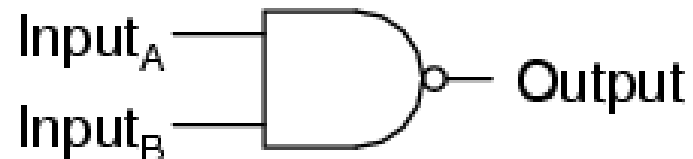


A	B
0	1
1	0

Types of Logic Gates

- This is a NOT-AND gate which is equal to an AND gate followed by a NOT gate.
- The outputs of all NAND gates are **high if any of the inputs are low**.
- Symbol is an AND gate with a small circle on the output. The small circle represents inversion.

NAND gate

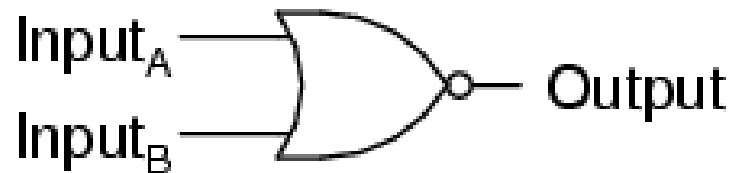


A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0

Types of Logic Gates

- This is a NOT-OR gate which is equal to an OR gate followed by a NOT gate.
- The outputs of all NOR gates **are low if any of the inputs are high.**
- The symbol is an OR gate with a small circle on the output. The small circle represents inversion.

NOR gate



A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0

Types of Logic Gates

- The '**Exclusive-OR**' **XOR** gate is a circuit which will give a **high output if either, but not both, of its two inputs are high**.
- An encircled plus sign ($A \oplus B$) is used to show the XOR operation.

Exclusive-OR gate

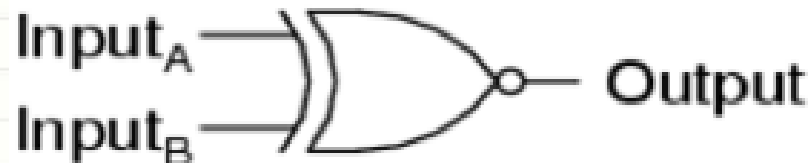


A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0

Types of Logic Gates

- The '**Exclusive-NOR**' **XNOR** gate circuit does the opposite to the EOR gate. It will give a **low output if either, but not both, of its inputs are high**.
- The symbol is an EXOR gate with a small circle on the output. The small circle represents inversion.

Exclusive-NOR gate

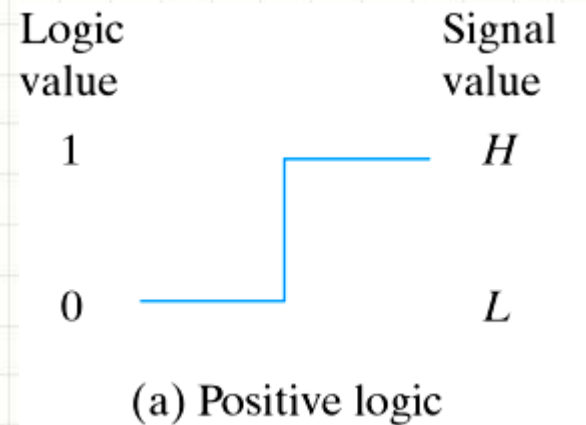


A	B	Output
0	0	1
0	1	0
1	0	0
1	1	1

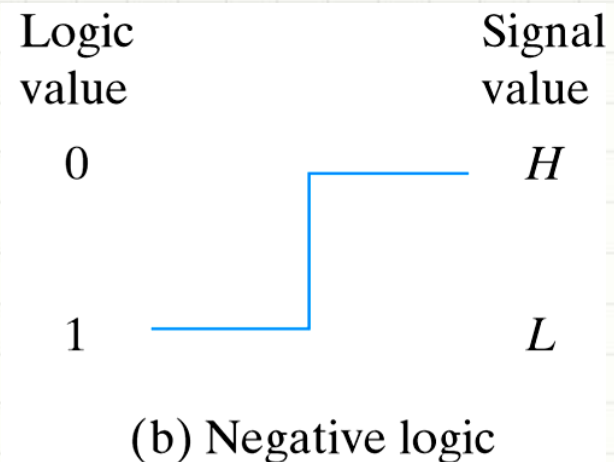
Positive and Negative Logic

Two signal values $\Leftrightarrow$ two logic values.

1. Positive logic: $H=1$; $L=0$



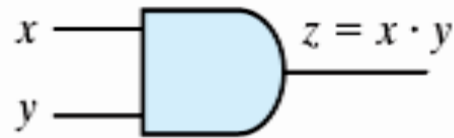
1. Negative logic: $H=0$; $L=1$



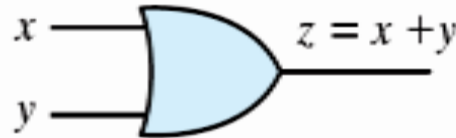
Binary Logic

- Logic gates

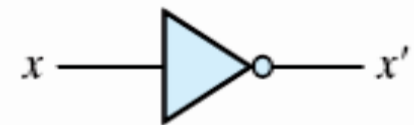
Symbols for digital logic circuits



(a) Two-input AND gate

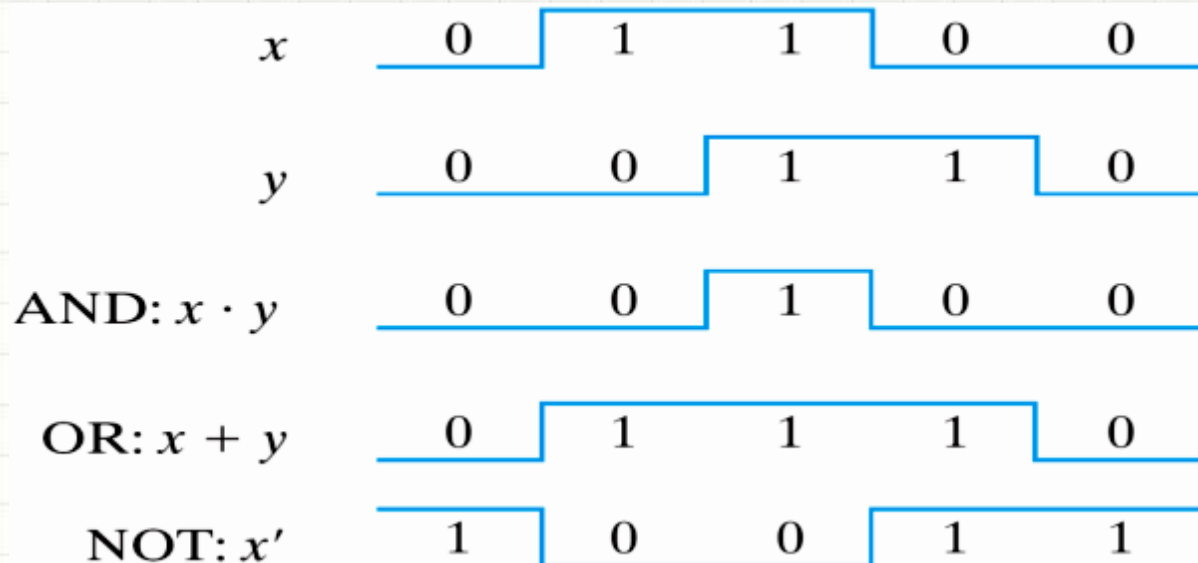


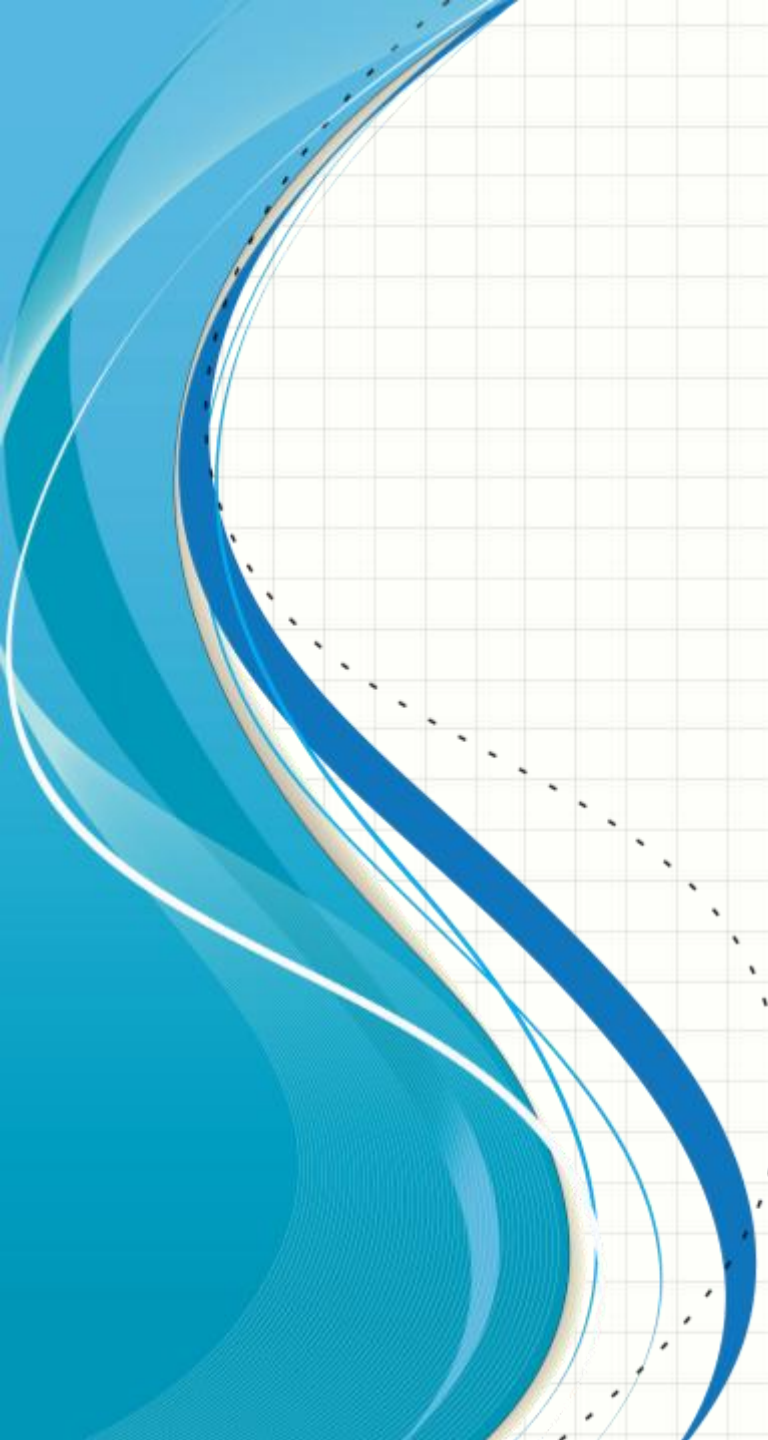
(b) Two-input OR gate



(c) NOT gate or inverter

Input-Output signals for gates





BOOLEAN ALGEBRA

Introduction

- **Boolean algebra** is the branch of **algebra** in which the values of the variables are the truth values true and false, usually denoted 1 and 0 respectively.
- As in standard algebra, Boolean algebra has 3 main elements:
 1. Constants: Power Supply Voltage (Logic 1)
 Ground Voltage (Logic 0)
 2. Variables: High = 1, Low = 0
 3. Operators: AND
 OR
 NOT
- Terminology:
 - *Literal*: A variable or its complement
 - *Product term*: literals connected by •
 - *Sum term*: literals connected by +

Basic Theorems

- (a) The structure is closed with respect to the operator $+$.
 - (b) The structure is closed with respect to the operator $\cdot$.
- (a) The element 0 is an identity element with respect to $+$; that is, $x + 0 = 0 + x = x$.
 - (b) The element 1 is an identity element with respect to $\cdot$; that is, $x \cdot 1 = 1 \cdot x = x$.
- (a) The structure is commutative with respect to $+$; that is, $x + y = y + x$.
 - (b) The structure is commutative with respect to $\cdot$; that is, $x \cdot y = y \cdot x$.
- (a) The operator $\cdot$ is distributive over $+$; that is, $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$.
 - (b) The operator $+$ is distributive over $\cdot$; that is, $x + (y \cdot z) = (x + y) \cdot (x + z)$.
- For every element $x \in B$, there exists an element $x' \in B$ (called the *complement* of x) such that (a) $x + x' = 1$ and (b) $x \cdot x' = 0$.
- There exist at least two elements $x, y \in B$ such that $x \neq y$.
- The distributive law of $+$ over $\cdot$ (i.e., $x + (y \cdot z) = (x + y) \cdot (x + z)$), is valid for Boolean algebra, but not for ordinary algebra.

Basic identity and Priorities

AND Identities

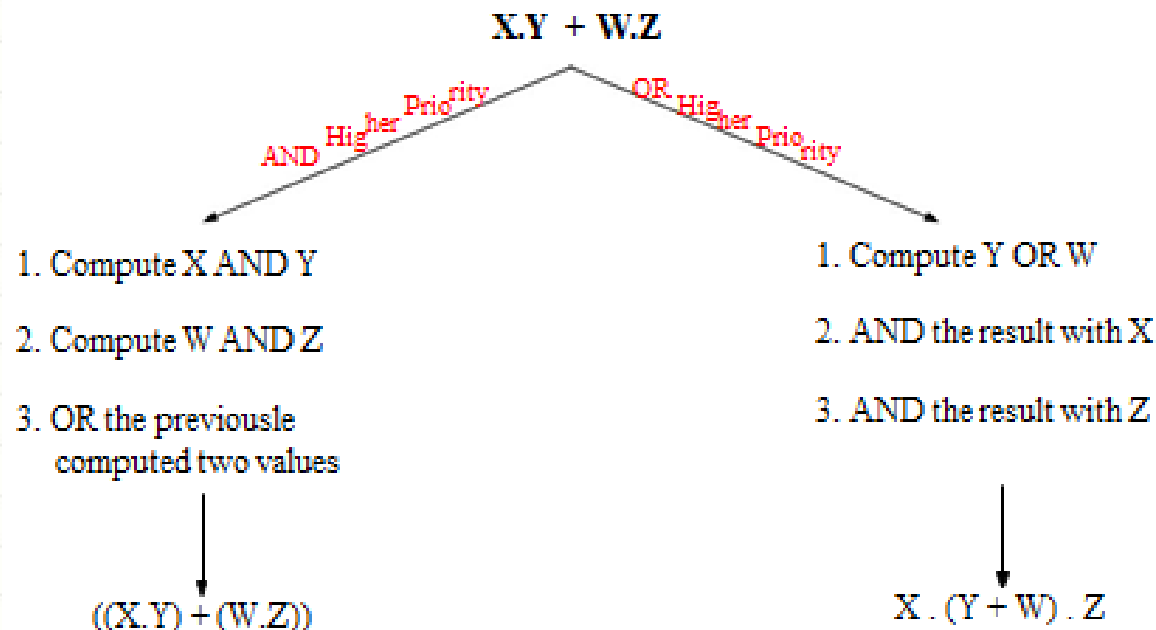
- $0 \cdot X = 0$
- $1 \cdot X = X$
- $X \cdot X = X$
- $X \cdot X' = 0$

OR Identities

- $1 + X = 1$
- $0 + X = X$
- $X + X = X$
- $X + X' = 1$

Priorities

- Parentheses *Highest Priority*
- Not operator (Complement)
- AND operator,
- OR operator *Lowest Priority*



Basic identities

Basic Identities of Boolean Algebra

$$(1) \ x + 0 = x$$

$$(3) \ x + 1 = 1$$

$$(5) \ x + x = x$$

$$(7) \ x + x' = 1$$

$$(9) \ x + y = y + x$$

$$(11) \ x + (y + z) = (x + y) + z$$

$$(13) \ x(y + z) = xy + xz$$

$$(15) \ (x + y)' = x'y'$$

$$(17) \ (x')' = x$$

$$(2) \ x \cdot 0 = 0$$

$$(4) \ x \cdot 1 = x$$

$$(6) \ x \cdot x = x$$

$$(8) \ x \cdot x' = 0$$

$$(10) \ xy = yx$$

$$(12) \ x(yz) = (xy)z$$

$$(14) \ x + yx = (x + y)(x + z)$$

$$(16) \ (xy)' = x' + y'$$

DeMorgan's Theorem

DeMorgan's theorem, $(x + y)' = x'y'$, is as follows:

x	y	$x + y$	$(x + y)'$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

x'	y'	$x'y'$
1	1	1
1	0	0
0	1	0
0	0	0

These theorems can be generalized as follows:

$$(A + B + C + D + \dots + F)' = A'B'C'D' \dots F'$$

$$(ABCD \dots F)' = A' + B' + C' + D' + \dots + F'$$

Complement of the functions *by* applying DeMorgan's theorem

Find the complement of the functions $F_1 = x'yz' + x'y'z$ and $F_2 = x(y'z' + yz)$. By applying DeMorgan's theorems obtained as follows:

$$F_1' = (x'yz' + x'y'z)' = (x'yz')'(x'y'z)' = (x + y' + z)(x + y + z')$$

$$\begin{aligned} F_2' &= [x(y'z' + yz)]' = x' + (y'z' + yz)' = x' + (y'z')'(yz)' \\ &= x' + (y + z)(y' + z') \\ &= x' + yz' + y'z \end{aligned}$$

Simplification to Minimum number of Literals

Simplify the following Boolean functions to a minimum number of literals.

1. $x(x' + y) = xx' + xy = 0 + xy = xy.$

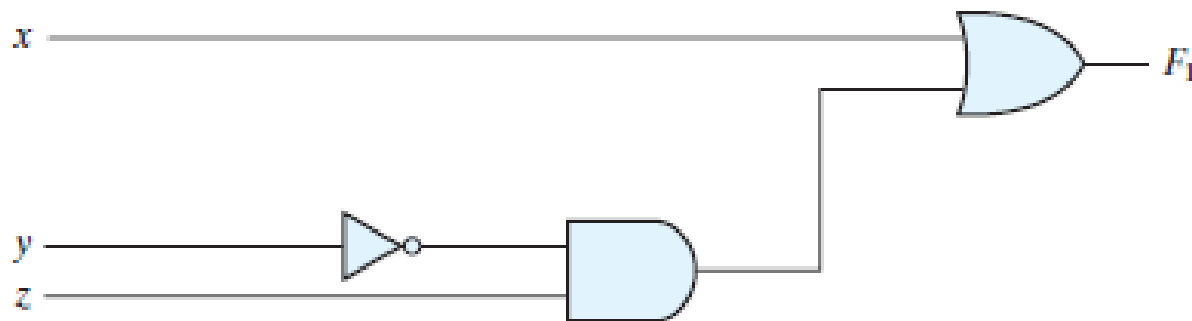
2. $x + x'y = (x + x')(x + y) = 1(x + y) = x + y.$

3. $(x + y)(x + y') = x + xy + xy' + yy' = x(1 + y + y') = x.$

4.
$$\begin{aligned} xy + x'z + yz &= xy + x'z + yz(x + x') \\ &= xy + x'z + xyz + x'yz \\ &= xy(1 + z) + x'z(1 + y) \\ &= xy + x'z. \end{aligned}$$

Implementation of Boolean Function with Gates

- $F_1 = x + y'z$



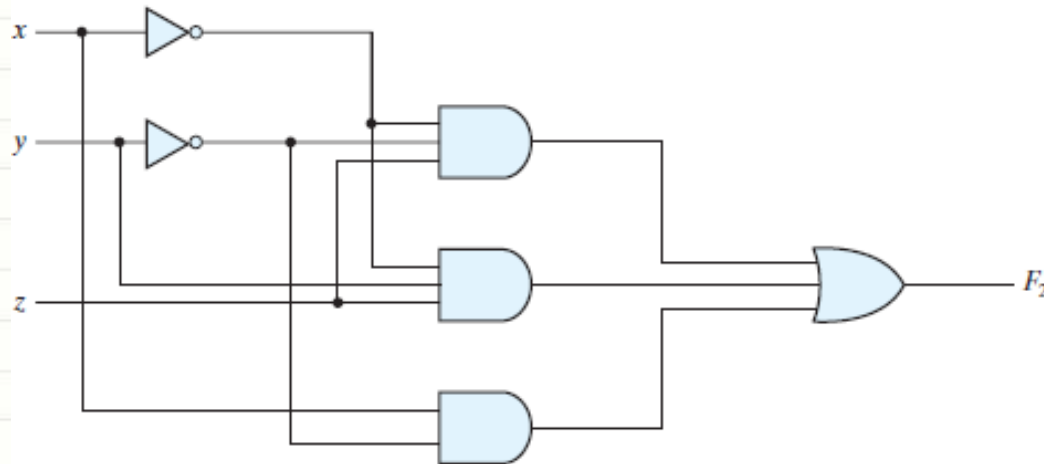
Gate implementation of $F_1 = x + y'z$

Truth Tables for F_1

x	y	z	F_1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Implementation of Boolean Function with Gates

$$F_2 = x'y'z + x'yz + xy'$$

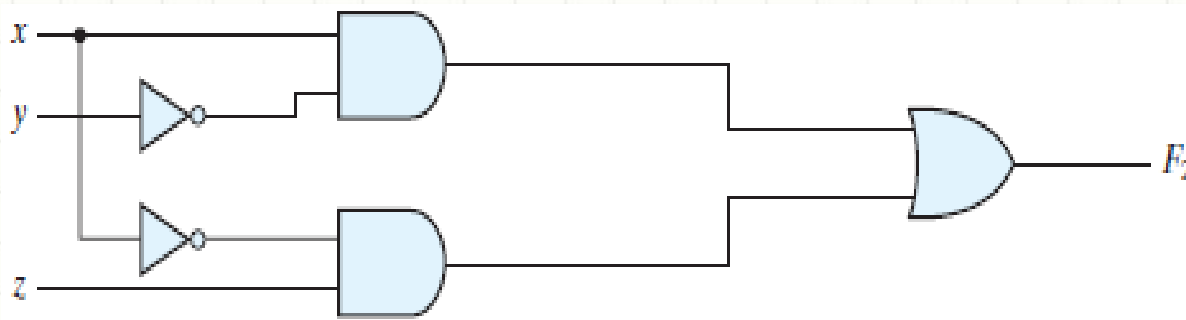


(a) $F_2 = x'y'z + x'yz + xy'$

Truth Tables for F_2

x	y	z	F_2
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$F_2 = x'y'z + x'yz + xy' = x'z(y' + y) + xy' = x'z + xy'$$



(b) $F_2 = xy' + x'z$

Minterms and Maxterms

Variable			Minterm		Maxterm	
x	y	z	Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x+y+z$	M_0
0	0	1	$x'y'z$	m_1	$x+y+z'$	M_1
0	1	0	$x'yz'$	m_2	$x+y'+z$	M_2
0	1	1	$x'yz$	m_3	$x+y'+z'$	M_3
1	0	0	$xy'z'$	m_4	$x'+y+z$	M_4
1	0	1	$xy'z$	m_5	$x'+y+z'$	M_5
1	1	0	xyz'	m_6	$x'+y'+z$	M_6
1	1	1	xyz	m_7	$x'+y'+z'$	M_7

Maxterm M_i is the complement of minterm m_i

Canonical Forms

Sum of Product (Minterms) SOP

- A function that is not in the Sum-of-Minterms form can be converted to that form by means of a truth table
- Consider $F = \bar{y} + \bar{x} \bar{z}$

x	y	z	F	Minterm
0	0	0	1	$m_0 = \bar{x} \bar{y} \bar{z}$
0	0	1	1	$m_1 = x \bar{y} \bar{z}$
0	1	0	1	$m_2 = \bar{x} y \bar{z}$
0	1	1	0	
1	0	0	1	$m_4 = x \bar{y} \bar{z}$
1	0	1	1	$m_5 = x y \bar{z}$
1	1	0	0	
1	1	1	0	

$$F = \sum(0, 1, 2, 4, 5) =$$

$$m_0 + m_1 + m_2 + m_4 + m_5 =$$

$$\bar{x} \bar{y} \bar{z} + \bar{x} \bar{y} z + \bar{x} y \bar{z} +$$

$$x \bar{y} \bar{z} + x y \bar{z}$$

Canonical Forms

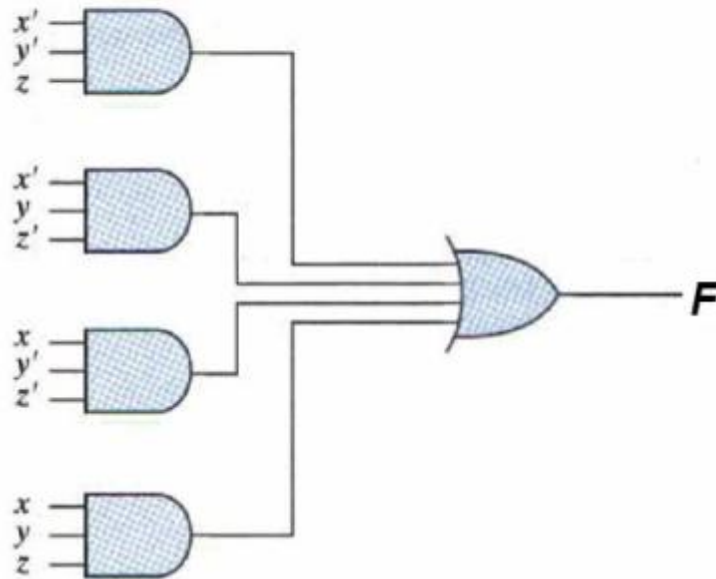
Product of sum (Maxterms) POS

- A function that is not in the Product-of-Minterms form can be converted to that form by means of a truth table
- Consider again: $F = \bar{y} + \bar{x} \bar{z}$

x	y	z	F	Minterm	$F = \prod(3, 6, 7) =$
0	0	0	1		
0	0	1	1		$M_3 \cdot M_6 \cdot M_7 =$
0	1	0	1		
0	1	1	0	$M_3 = (x+y+z)^-$	$(x+\bar{y}+\bar{z}) (\bar{x}+\bar{y}+z) (\bar{x}+\bar{y}+\bar{z})$
1	0	0	1		
1	0	1	1		
1	1	0	0	$M_6 = (x\bar{y}\bar{z})$	
1	1	1	0	$M_7 = (x\bar{y}z)^-$	

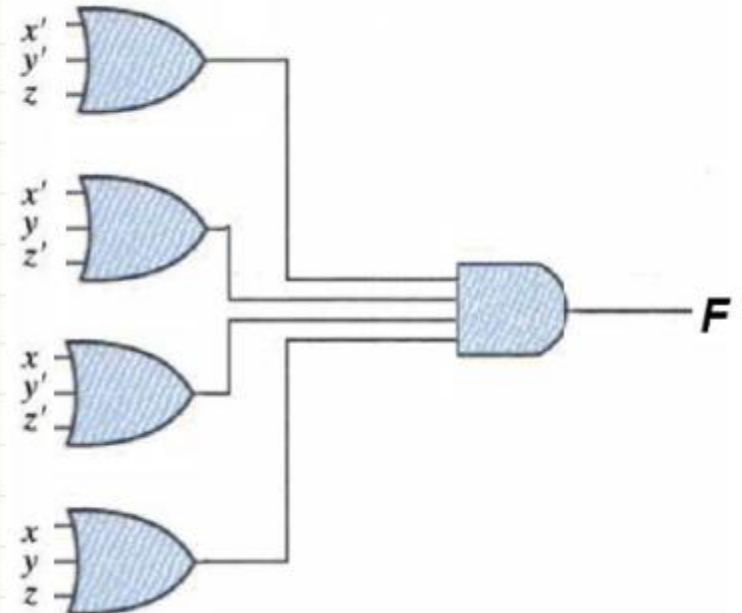
Two Level Implementation

$$F = x'y'z + x'yz' + xy'z' + xyz$$

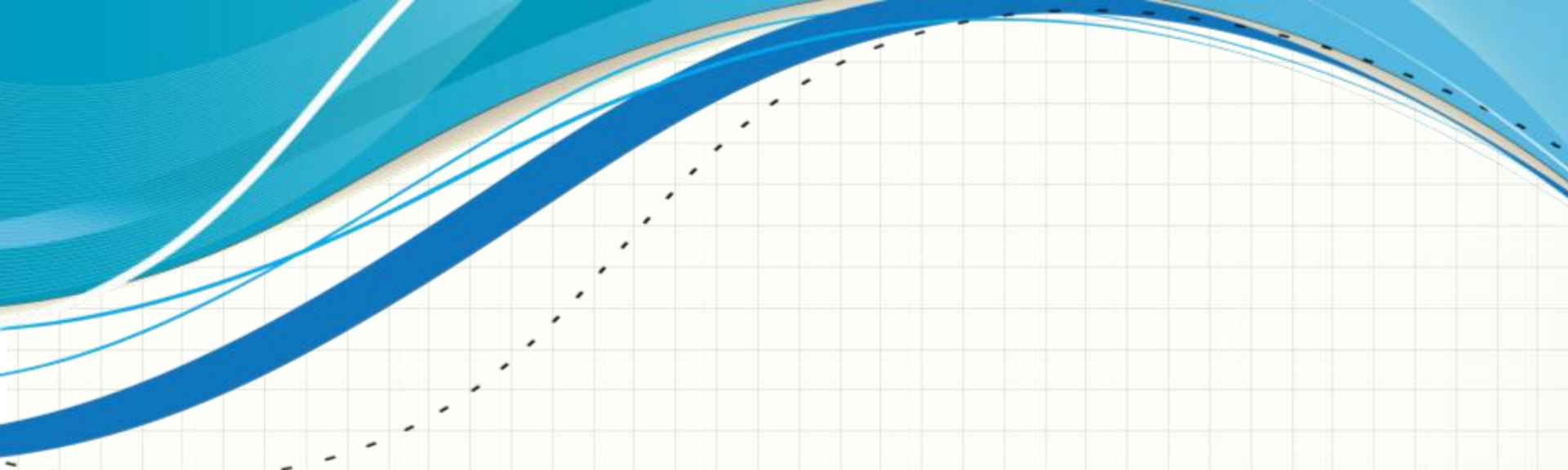


Sum of Product (SOP)

$$F = (x' + y' + z)(x' + y + z')(x + y' + z')(x + y + z)$$



Product of Sum (POS)



THANKS